

WEB DEVELOPMENT

Backend Development

Hyunwoo Park

Graduate School of Data Science

Seoul National University

hyunwoopark@snu.ac.kr / <https://hyunwoopark.com>



Today's Agenda

Understanding the Backend:

- **What is Backend?** — The invisible layer that powers everything
- **History of Backend** — CGI to serverless, 30 years of evolution
- **Full Stack Architecture** — How frontend, backend, database connect
- **Backend Frameworks Landscape** — What's out there and how to choose
- **REST API Design** — The universal language of web communication

After this lecture, you will be able to:

- Explain why backends exist and what problems they solve
- Trace the evolution of backend technology from CGI to serverless
- Compare major backend frameworks and their trade-offs
- Design RESTful API endpoints



Part 1

What is Backend?

Frontend vs. Backend — A Restaurant Analogy

Frontend = Dining Room

- What customers see: tables, menus, decor
- Waiter takes your order, brings your food
- Beautiful, customer-facing

Backend = Kitchen

- What customers never see: ovens, fridges, prep stations
- Chef cooks the food, manages ingredients
- Where the real work happens

Database = Pantry / Supply Room

- Stores raw ingredients (data)
- Organized for quick retrieval
- Restocked regularly

A restaurant with a beautiful dining room but no kitchen serves nothing.

What Does a Backend Actually Do?

1. Store and retrieve data

User clicks "My Reports" → Backend queries database → Returns user's reports

2. Enforce business logic

User tries to withdraw \$1000 → Backend checks: balance $\geq$ 1000? → Allow or deny

3. Authenticate users

User logs in → Backend verifies password → Issues session token

4. Protect sensitive operations

Frontend NEVER touches the database directly
All data flows through the backend

5. Integrate with external services

Send emails, process payments, call other APIs

Why Not Just Use Frontend?

Can't you do everything in JavaScript in the browser?

Security problem:

```
// Frontend code is PUBLIC – anyone can read it
// Open DevTools → Sources → see all your code

// WRONG: checking payment in frontend
if (user.balance >= amount) { processPayment(); }
// User hacks: DevTools → Console → user.balance = 999999

// RIGHT: checking payment in backend
// Server has the REAL balance, user can't modify it
```

Data problem:

- Where do you store user data if there's no server?
- Browser storage (localStorage) is per-device, temporary
- Need persistent, shared storage → need a server

Integration problem:

- API keys for Stripe, SendGrid, etc. must be secret
Can't put secrets in frontend code (everyone can see it)



Part 2

History of Backend Development

The Evolution — 30 Years in 5 Minutes

1993	CGI Scripts	→ Perl/C programs, one process per request
1995	PHP	→ "Personal Home Page", embedded in HTML
1996	Java Servlets	→ Enterprise, verbose, but reliable
2004	Ruby on Rails	→ "Convention over configuration", developer joy
2005	Django	→ Python's "batteries included" framework
2009	Node.js	→ JavaScript on the server, event-driven
2010	Flask	→ Python minimalist micro-framework
2012	Go (net/http)	→ Google's systems language, fast compilation
2018	FastAPI	→ Python + type hints + async, auto-docs
2020+	Serverless	→ No server management, pay per request

The trend: From low-level to high-level, from verbose to concise, from server-managed to serverless.

Era 1: CGI (1993–2000)

CGI = Common Gateway Interface

Browser → Web Server (Apache) → Fork new process → Run Perl script → Return HTML

How it worked:

- Each request spawns a new process
- Script generates HTML as a string
- Slow, resource-intensive

```
#!/usr/bin/perl
print "Content-Type: text/html\n\n";
print "<html><body>";
print "<h1>Hello, World!</h1>";
print "<p>Current time: " . localtime() . "</p>";
print "</body></html>";
```

Problems: One process per request = server crashes at 100 users. No state management. Spaghetti code mixing HTML and logic.

Legacy: The concept of "web application" was born here.

• **What remained unsolved:** Too slow. Embedding logic in HTML was a mess. Need something better.

Era 2: PHP and the Dynamic Web (1995–2010)

PHP = "Personal Home Page" → "PHP: Hypertext Preprocessor"

```
<html>
<body>
  <h1>Welcome, <?php echo $username; ?>!/h1>
  <?php
    $result = mysql_query("SELECT * FROM posts");
    while ($row = mysql_fetch_array($result)) {
      echo "<p>" . $row['title'] . "</p>";
    }
  ?>
</body>
</html>
```

Revolution: Embed code directly in HTML. Apache mod_php = no process forking. MySQL integration built-in.

Impact: WordPress (2003), Facebook (2004), Wikipedia — PHP powered the early social web.

77%

of websites with a known server-side language still use PHP (W3Techs, 2025)

WordPress, Wikipedia, Facebook all started here.

→ **What remained unsolved:** No architecture. No separation of concerns. Code and HTML tangled together.

Era 3: Frameworks — Rails and Django (2004–2010)

Ruby on Rails (2004) — "Convention over Configuration"

```
# One command creates a full CRUD app:  
rails generate scaffold Post title:string body:text
```

Django (2005) — "The Web Framework for Perfectionists with Deadlines"

```
# models.py — define your data  
class Post(models.Model):  
    title = models.CharField(max_length=200)  
    body = models.TextField()  
    created_at = models.DateTimeField(auto_now_add=True)  
  
# That's it. Django auto-generates admin panel, database tables, forms.
```

What changed:

- MVC/MTV architecture (separation of concerns)
- ORM (Object-Relational Mapping) — write Python, not SQL
- Built-in admin panels, authentication, migrations
- "Don't Repeat Yourself" (DRY) principle

Era 4: Node.js and the JavaScript Everywhere (2009+)

Node.js — JavaScript on the Server

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('<h1>Hello from Node.js!</h1>');
});

server.listen(3000);
```

Why it mattered:

- **Same language** for frontend AND backend (JavaScript everywhere)
- **Event-driven, non-blocking I/O** — handles thousands of concurrent connections
- **npm** — largest package ecosystem in any language (2M+ packages)

Express.js (2010) became the de facto Node.js framework:

```
const express = require('express');
const app = express();
```

2M+

packages on npm (Node.js package manager)

The largest package ecosystem of any programming language.

JavaScript went from "browser toy" to "runs everything" in 15 years.

Era 5: Modern Python — FastAPI (2018+)

FastAPI = Python + Type Hints + Async + Auto-Documentation

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    email: str

@app.get("/users/{user_id}")
async def get_user(user_id: int):
    return {"user_id": user_id, "name": "Alice"}

@app.post("/users")
async def create_user(user: User):
    return {"id": 1, **user.model_dump()}
```

What makes it special:

- Type hints → automatic validation + documentation
- Visit `/docs` → interactive Swagger UI for free
- Async-native → handles concurrent requests efficiently

Era 6: Serverless and Beyond (2020+)

Serverless = "No server to manage"

```
# AWS Lambda / Cloudflare Workers / Vercel Functions
def handler(event, context):
    name = event.get("name", "World")
    return {"statusCode": 200, "body": f"Hello, {name}!"}
```

How it works:

- Write a function, deploy it
- Cloud provider runs it when called
- Pay only for execution time (not idle server)
- Auto-scales from 0 to millions of requests

Backend-as-a-Service (BaaS):

- **Firebase** (Google) — real-time database + auth + hosting
- **Supabase** — open-source Firebase alternative (PostgreSQL)
- **PocketBase** — single-binary backend (Go)

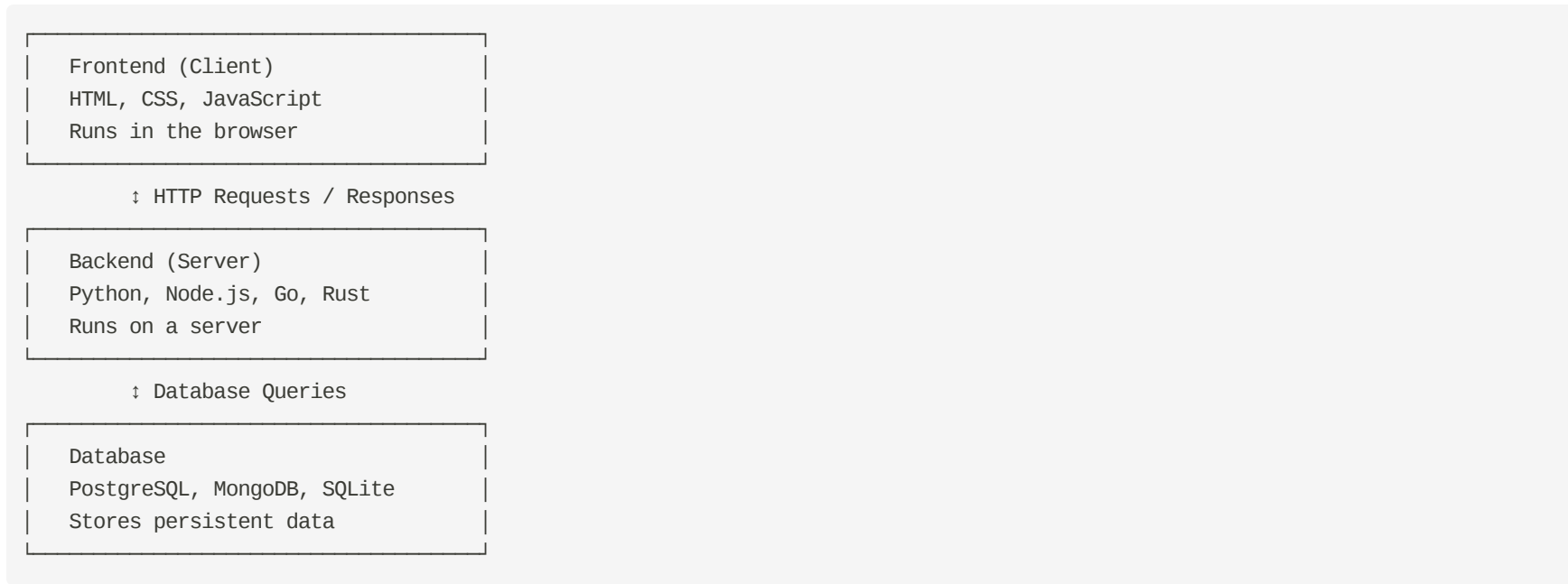
The trend: Backend is becoming invisible. Focus on logic, not infrastructure.



Part 3

Full Stack Architecture

The Three-Layer Cake



Every user action: Click → HTTP request → Backend processes → Database query → Response → UI update

Client-Server Communication

A login flow, step by step:

```
1. User types email + password (frontend)
  ↓
2. Frontend sends: POST /api/login
   Body: {"email": "user@snu.ac.kr", "password": "..."}
  ↓
3. Backend receives request
   → Looks up email in database
   → Verifies password hash
   → Creates session token
  ↓
4. Backend responds: 200 OK
   {"token": "eyJhbG...", "user": {"name": "Alice"}}
  ↓
5. Frontend stores token in browser
  ↓
6. Next request: GET /api/reports
   Header: Authorization: Bearer eyJhbG...
  ↓
7. Backend verifies token → queries database → returns reports
```

Key: Frontend NEVER touches the database. All data flows through backend.

Why Separate Frontend and Backend?

Security:

- Database credentials stay on the server (never in browser)
- Business logic is hidden from users
- API keys for external services are protected

Scalability:

- Frontend on CDN → fast, global, cheap
- Backend auto-scales → handle traffic spikes
- Database behind firewall → secure

Flexibility:

- Same backend serves web app + mobile app + CLI tool
- Replace React with Vue without touching backend
- Swap PostgreSQL for MongoDB without frontend changes

Team independence:

- Frontend team works on UI/UX
- Backend team works on data/logic
- They communicate through the API contract

Static Sites vs. Dynamic Apps

Static site (what you built last week):

GitHub Pages → serves HTML/CSS/JS files → same content for everyone
No server, no database, no login → fast, free, simple

Dynamic app (what we'll build this week):

Browser → Backend server → Database → personalized response
Different content per user → needs server + database

When to use which:

Feature	Static Site	Dynamic App
Blog, portfolio	✓	
User accounts		✓
Data storage		✓



Part 4

Backend Frameworks Landscape

**The core problem never
changed.**

Only the tools did.

CGI → PHP → Rails/Django → Node.js → FastAPI → Serverless

Each era solved the previous era's pain — and created new ones.

Choosing a Backend Framework

Quick exercise (2 minutes): Think about this before I show the answer.

You're building a data visualization dashboard that serves D3.js charts from a Python data pipeline. Which framework would you pick and why?

Turn to your neighbor and share your reasoning.

Framework Landscape

The big question: Which language? Which framework?

There is no "best" framework — only the best fit for your situation.

Factors to consider:

Your team's language — Use what you know

Performance needs — How many requests per second?

Ecosystem — Libraries, community, hiring market

Learning curve — How fast can you ship?

Type of project — Prototype vs. enterprise

Python Frameworks

Framework	Style	Best For	Learning Curve
Django	Batteries-included	Full apps with admin	Medium-Hard
Flask	Minimalist	Small APIs, prototypes	Easy
FastAPI	Modern, async	APIs with docs	Medium

Django: Everything built-in (admin, auth, ORM, forms). Heavy but complete.

Flask: Minimal core, add what you need. Simple but you build everything.

FastAPI: Modern sweet spot. Auto-docs, validation, async. Our choice for this course.

Flask:

```
from flask import Flask
app = Flask(__name__)

@app.route('/hello')
def hello():
    return {"msg": "hi"}
```

Run: `flask run`

FastAPI:

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/hello")
def hello():
    return {"msg": "hi"}
```

Run: `uvicorn main:app --reload`

Plus: auto-docs at `/docs` for free!

JavaScript / TypeScript Frameworks

Framework	Style	Best For
Express	Minimalist	Most Node.js projects
Next.js	Full-stack React	React apps with API routes
Hono	Ultra-fast	Edge/serverless
NestJS	Enterprise	Large TypeScript projects

Express: The jQuery of backend. Everyone knows it. Simple.

```
const express = require('express');
const app = express();

app.get('/hello', (req, res) => {
  res.json({ msg: 'hi' });
});

app.listen(3000);
```

Other Languages

Go:

```
// Minimal, fast, great for microservices
http.HandleFunc("/hello", func(w http.ResponseWriter, r *http.Request) {
    json.NewEncoder(w).Encode(map[string]string{"msg": "hi"})
})
http.ListenAndServe(":8080", nil)
```

- Compiled, extremely fast. Used at Google, Uber, Twitch. - Simple language, easy to learn, but verbose.

Rust (Axum/Actix):

- Fastest benchmarks. Memory-safe without garbage collector.
- Steep learning curve. Used when performance is critical.

Java (Spring Boot):

- Enterprise standard. Banks, insurance, government.
- Verbose but battle-tested. Massive ecosystem.

Each language has its niche. For data science, Python is the natural choice.

Framework Decision Matrix

Factor	Django	Flask	FastAPI	Express	Next.js	Go
Speed	★★	★★★★	★★★★★	★★★★	★★★	★★★★★★
Ease	★★★★	★★★★★★	★★★★★	★★★★★	★★★	★★★★
Docs	★★★★★	★★★★	★★★★★★	★★★★	★★★★★	★★★★
Ecosystem	★★★★★★	★★★★	★★★★	★★★★★★	★★★★★	★★★★
Data Science	★★★★★	★★★★★	★★★★★★	★★	★★	★

Our choice: FastAPI

- Python (you already know it)
- Auto-documentation (great for learning APIs)
- Modern async support
- Perfect for data science projects



Part 5

REST API Design

What is an API?

API = Application Programming Interface

A contract between two systems: "If you send me this, I'll send you that."

Analogy — a vending machine:

- You insert coins (request)
- Press a button (endpoint)
- Machine gives you a drink (response)
- You don't need to know how the machine works inside

Web API:

- Frontend sends HTTP request (with data)
- Backend processes it (runs logic, queries database)
- Backend returns HTTP response (with data)
- Frontend doesn't need to know backend implementation

REST Principles

REST = Representational State Transfer

Core principles:

Resources — Everything is a resource (user, report, patient)

URLs identify resources — `/users/5` , `/reports/42`

HTTP verbs define actions — GET, POST, PUT, DELETE

Stateless — Each request is self-contained

JSON for data — Universal format

REST is a style, not a standard. It's how 99% of modern APIs work.

HTTP Verbs (Methods)

Verb	Action	Example	SQL Equivalent
GET	Read	GET /users/5	SELECT
POST	Create	POST /users	INSERT
PUT	Replace	PUT /users/5	UPDATE
DELETE	Remove	DELETE /users/5	DELETE

Same URL, different verb = different action:

```
GET    /users    → List all users
POST   /users    → Create a new user
GET    /users/5   → Get user #5
PUT    /users/5   → Update user #5
DELETE /users/5   → Delete user #5
```

In practice: Many APIs simplify to GET + POST only.

URL Design

Good (REST-style):

```
GET    /api/v1/patients      → List patients
POST   /api/v1/patients      → Create patient
GET    /api/v1/patients/42   → Get patient 42
GET    /api/v1/patients/42/vitals → Get patient 42's vitals
```

Bad (RPC-style):

```
GET    /getPatientById?id=42
POST    /createNewPatient
POST    /updatePatientData
```

Rules:

- Plural nouns (`/patients` , not `/patient`)
- IDs in the path (`/patients/42` , not `?id=42`)
- Nest related resources (`/patients/42/vitals`)
- Version your API (`/api/v1/...`)

JSON — The Language of APIs

Request:

```
POST /api/v1/patients
Content-Type: application/json
Authorization: Bearer eyJhbG...
```

```
{
  "name": "김철수",
  "age": 45,
  "department": "cardiology",
  "admitted": true
}
```

Response:

```
HTTP/1.1 201 Created
Content-Type: application/json
```

```
{
  "id": 42,
  "name": "김철수",
  "age": 45,
  "department": "cardiology",
  "admitted": true,
```

HTTP Status Codes

The backend tells the frontend what happened:

Code	Meaning	When to use
200	OK	Successful GET or PUT
201	Created	Successful POST
204	No Content	Successful DELETE
400	Bad Request	Invalid input
401	Unauthorized	Not logged in
403	Forbidden	Logged in but no permission
404	Not Found	Resource doesn't exist

API Design Best Practices

1. Pagination:

```
GET /api/v1/patients?page=1&limit=20  
→ Returns patients 1-20 (not all 10,000)
```

2. Filtering:

```
GET /api/v1/patients?department=cardiology&admitted=true  
→ Only cardiology patients who are admitted
```

3. Error responses (structured):

```
{  
  "error": "Validation Error",  
  "details": [  
    {"field": "email", "message": "Invalid email format"},  
    {"field": "age", "message": "Must be a positive integer"}  
  ]  
}
```

4. Documentation:

Quick Demo: FastAPI in Action

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# Data model
class Patient(BaseModel):
    name: str
    age: int
    department: str

# In-memory storage (Thursday: we'll use a real database)
patients = []

@app.get("/api/v1/patients")
def list_patients():
    return {"patients": patients}

@app.post("/api/v1/patients", status_code=201)
def create_patient(patient: Patient):
    record = {"id": len(patients) + 1, **patient.model_dump()}
    patients.append(record)
    return record
```

Key Takeaways

Backend exists because:

- Security — sensitive logic must stay on the server
- Data — persistent storage needs a server + database
- Integration — external services need secret API keys

30 years of evolution:

- CGI → PHP → Rails/Django → Node.js → FastAPI → Serverless
- Trend: simpler, faster, more developer-friendly

Architecture:

- Frontend + Backend + Database = Full Stack
- Frontend never touches the database directly
- API is the contract between frontend and backend

REST API design:

- Resources + HTTP verbs + JSON
- `/api/v1/patients/42` — clean, predictable, standard

Thursday: We'll build a real backend with FastAPI and connect it to your frontend.

Thank you.

Backend Development

Hyunwoo Park
Graduate School of Data Science
Seoul National University
hyunwoopark@snu.ac.kr / <https://hyunwoopark.com>

